

DATA MODEL BASIC BUILDING BLOCKS

The basic building blocks of all data models are entities, attributes, relationships, and constraints. An **entity** is anything (a person, a place, a thing, or an event) about which data are to be collected and stored. An entity represents a particular type of object in the real world. Because an entity represents a particular type of object, entities are “distinguishable”—that is, each entity occurrence is unique and distinct. For example, a CUSTOMER entity would have many distinguishable customer occurrences, such as John Smith, Pedro Dinamita, Tom Strickland, etc. Entities may be physical objects, such as customers or products, but entities may also be abstractions, such as flight routes or musical concerts.

An **attribute** is a characteristic of an entity. For example, a CUSTOMER entity would be described by attributes such as customer last name, customer first name, customer phone, customer address, and customer credit limit. Attributes are the equivalent of fields in file systems.

A **relationship** describes an association among entities. For example, a relationship exists between customers and agents that can be described as follows: an agent can serve many customers, and each customer may be served by one agent. Data models use three types of relationships: one-to-many, many-to-many, and one-to-one. Database designers usually use the shorthand notations 1:M or 1..*, M:N or *.* , and 1:1 or 1..1, respectively. (Although the M:N notation is a standard label for the many-to-many relationship, the label M:M may also be used.)

- **One-to-many (1:M or 1..*) relationship.** A painter paints many different paintings, but each one of them is painted by only one painter. Thus, the painter (the “one”) is related to the paintings (the “many”). Therefore, database designers label the relationship “PAINTER paints PAINTING” as 1:M. (Note that entity names are often capitalized as a convention, so they are easily identified.) Similarly, a customer (the “one”) may generate many invoices, but each invoice (the “many”) is generated by only a single customer. The “CUSTOMER generates INVOICE” relationship would also be labeled 1:M.
- **Many-to-many (M:N or *.*) relationship.** An employee may learn many job skills, and each job skill may be learned by many employees. Database designers label the relationship “EMPLOYEE learns SKILL” as M:N. Similarly, a student can take many classes and each class can be taken by many students, thus yielding the M:N relationship label for the relationship expressed by “STUDENT takes CLASS.”
- **One-to-one (1:1 or 1..1) relationship.** A retail company’s management structure may require that each of its stores be managed by a single employee. In turn, each store manager, who is an employee, manages only a single store. Therefore, the relationship “EMPLOYEE manages STORE” is labeled 1:1.

The preceding discussion identified each relationship in both directions; that is, relationships are bidirectional:

- One CUSTOMER can generate *many* INVOICES.
- Each of the *many* INVOICES is generated by only *one* CUSTOMER.

A **constraint** is a restriction placed on the data. Constraints are important because they help to ensure data integrity. Constraints are normally expressed in the form of rules. For example:

- An employee’s salary must have values that are between 6,000 and 350,000.
- A student’s GPA must be between 0.00 and 4.00.
- Each class must have one and only one teacher.

How do you properly identify entities, attributes, relationships, and constraints? The first step is to clearly identify the business rules for the problem domain you are modeling.

BUSINESS RULES

From a database point of view, the collection of data becomes meaningful only when it reflects properly defined *business rules*. A **business rule** is a brief, precise, and unambiguous description of a policy, procedure, or principle within a specific organization. In a sense, business rules are misnamed: they apply to *any* organization, large or small—a business, a government unit, a religious group, or a research laboratory—that stores and uses data to generate information.

Business rules, derived from a detailed description of an organization’s operations, help to create and enforce actions within that organization’s environment. Business rules must be rendered in writing and updated to reflect any change in the organization’s operational environment.

Properly written business rules are used to define entities, attributes, relationships, and constraints. Any time you see relationship statements such as “an agent can serve many customers, and each customer can be served by only one agent,” you are seeing business rules at work.

Business rules describe, in simple language, the main and distinguishing characteristics of the data *as viewed by the company*. Examples of business rules are as follows:

- A customer may generate many invoices.
- An invoice is generated by only one customer.
- A training session cannot be scheduled for fewer than 10 employees or for more than 30 employees.

Note that those business rules establish entities, relationships, and constraints. For example, the first two business rules establish two entities (CUSTOMER and INVOICE) and a 1:M relationship between those two entities. The third business rule establishes a constraint (no fewer than 10 people and no more than 30 people), two entities (EMPLOYEE and TRAINING), and a relationship between EMPLOYEE and TRAINING.

Discovering Business Rules

A faster and more direct source of business rules is direct interviews with end users. Unfortunately, because perceptions differ, end users are sometimes a less reliable source when it comes to specifying business rules. For example, a maintenance department mechanic might believe that any mechanic can initiate a maintenance procedure, when actually only mechanics with inspection authorization can perform such a task. Such a distinction might seem trivial, but it can have major legal consequences.

The database designer's job is to reconcile any differences and verify the results of the reconciliation to ensure that the business rules are appropriate and accurate.

The process of identifying and documenting business rules is essential to database design for several reasons:

- They help to standardize the company's view of data.
- They can be a communications tool between users and designers.
- They allow the designer to understand the nature, role, and scope of the data.
- They allow the designer to understand business processes.
- They allow the designer to develop appropriate relationship participation rules and constraints and to create an accurate data model.

Not all business rules can be modeled. For example, a business rule that specifies that "no pilot can fly more than 10 hours within any 24-hour period" cannot be modeled. However, such a business rule can be enforced by application software.

As a general rule, a noun in a business rule will translate into an entity in the model, and a verb (active or passive) associating nouns will translate into a relationship among the entities. For example, the business rule "a customer may generate many invoices" contains two nouns (*customer* and *invoices*) and a verb (*generate*) that associates the nouns. From this business rule, you could deduce that:

- Customer and invoice are objects of interest for the environment and should be represented by their respective entities.
- There is a "generate" relationship between customer and invoice.

To properly identify the type of relationship, you should consider that relationships are bidirectional; that is, they go both ways. For example, the business rule "a customer may generate many invoices" is complemented by the business rule "an invoice is generated by only one customer." In that case, the relationship is one-to-many (1:M). Customer is the "1" side, and invoice is the "many" side.

As a general rule, to properly identify the relationship type, you should ask two questions:

- How many instances of B are related to one instance of A?
- How many instances of A are related to one instance of B?

For example, you can assess the relationship between student and class by asking two questions:

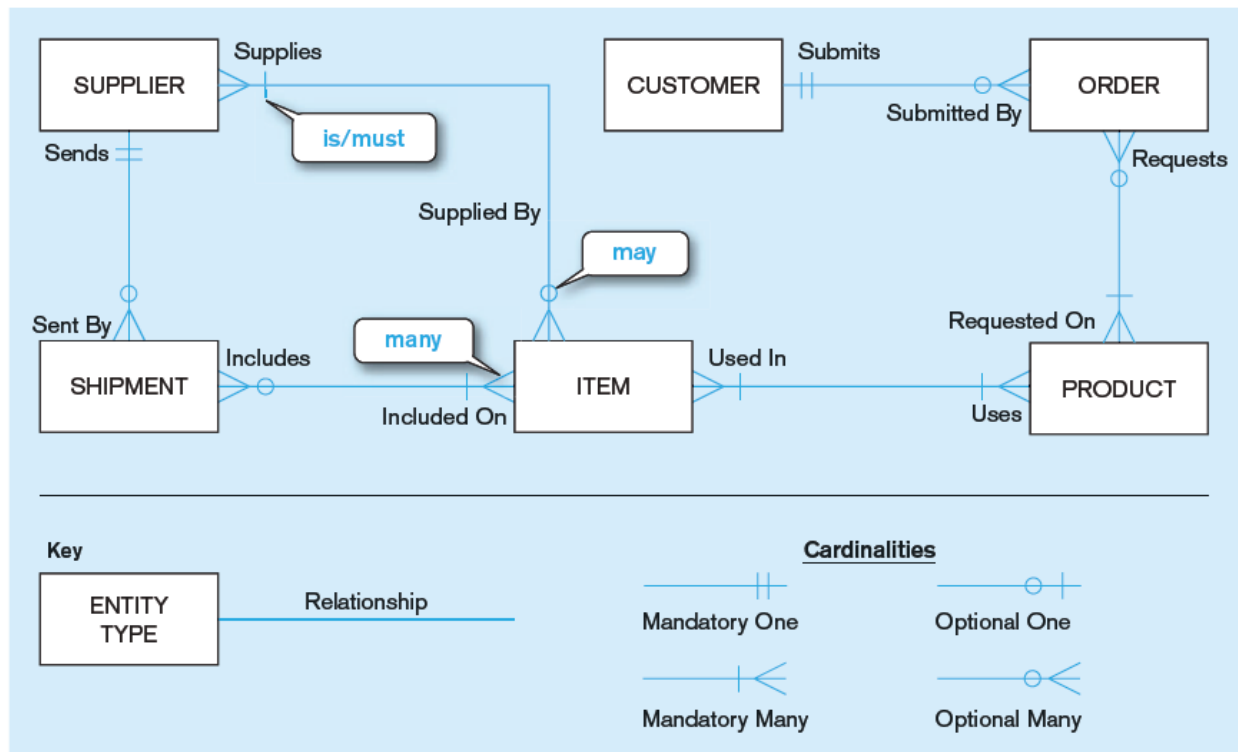
- In how many classes can one student enroll? Answer: many classes.
- How many students can enroll in one class? Answer: many students.

Therefore, the relationship between student and class is many-to-many (M:N).

NAMING CONVENTIONS

Entity names should be descriptive of the objects in the business environment, and use terminology that is familiar to the users. An attribute name should also be descriptive of the data represented by that attribute. It is also a good practice to prefix the name of an attribute with the name of the entity (or an abbreviation of the entity name) in which it occurs. For example, in the CUSTOMER entity, the customer's credit limit may be called CUS_CREDIT_LIMIT. The CUS indicates that the attribute is descriptive of the CUSTOMER entity, while CREDIT_LIMIT makes it easy to recognize the data that will be contained in the attribute.

Sample ER model



1. A SUPPLIER may supply many ITEMS (by "may supply," we mean the supplier may not supply any items). Each ITEM is supplied by any number of SUPPLIERS (by "is supplied," we mean that the item must be supplied by at least one supplier). See annotations in the Figure above that correspond to underlined words.
2. Each ITEM must be used in the assembly of at least one PRODUCT and may be used in many products. Conversely, each PRODUCT must use one or more ITEMS.
3. A SUPPLIER may send many SHIPMENTS. However, each shipment must be sent by exactly one SUPPLIER. Notice that sends and supplies are separate concepts. A SUPPLIER may be able to supply an item, but may not yet have sent any shipments of that item.
4. A SHIPMENT must include one (or more) ITEMS. An ITEM may be included on several SHIPMENTS.
5. A CUSTOMER may submit any number of ORDERS. However, each ORDER must be submitted by exactly one CUSTOMER. Given that a CUSTOMER may not have submitted any ORDERS, some CUSTOMERS must be potential, inactive, or some other customer possibly without any related ORDERS.
6. An ORDER must request one (or more) PRODUCTS. A given PRODUCT may not be requested on any ORDER, or may be requested on one or more orders.

There are actually two business rules for each relationship, one for each direction from one entity to the other. Note that each of these business rules roughly follows a certain grammar:

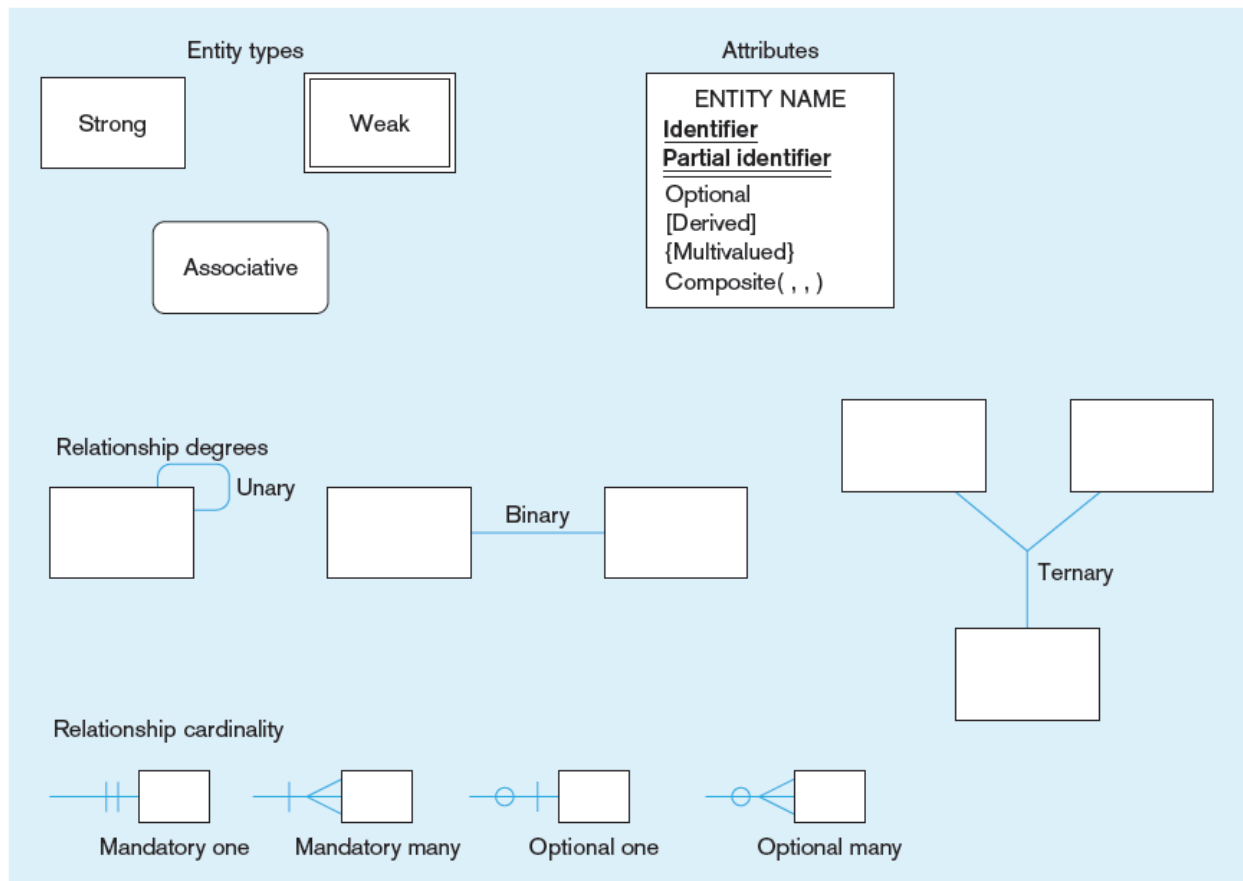
<entity> <minimum cardinality> <relationship> <maximum cardinality> <entity>

For example, rule 5 is:

<CUSTOMER> <may> <Submit> <any number> <ORDER>

This grammar gives you a standard way to put each relationship into a natural English business rule statement.

Basic E-R notation



NOTE: Research more on business rules, their scope, gathering and properties of good business rules

MODELING ENTITIES AND ATTRIBUTES

An **entity** is a person, a place, an object, an event, or a concept in the user environment about which the organization wishes to maintain data.

E.g. *Person*: EMPLOYEE, STUDENT, PATIENT

Place: STORE, WAREHOUSE, STATE

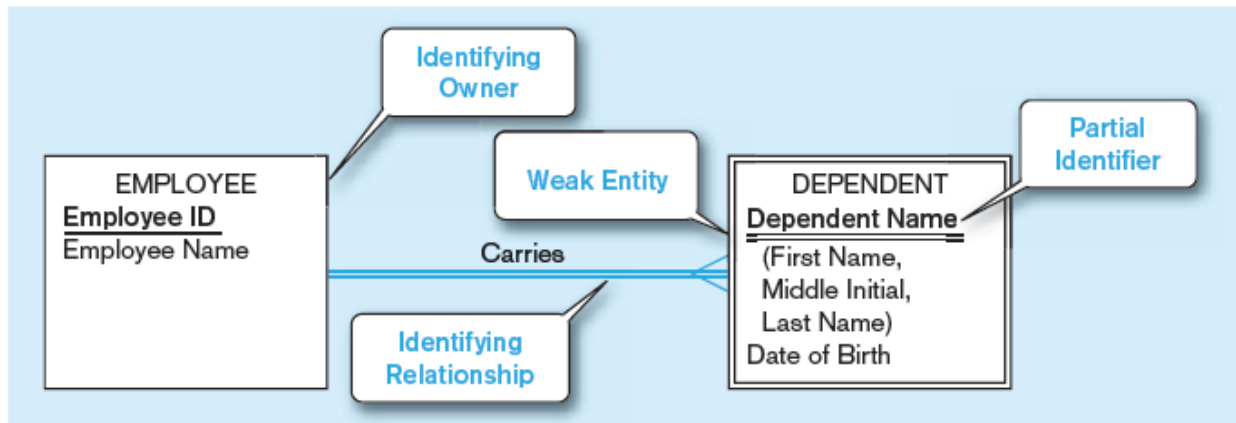
Object: MACHINE, BUILDING, AUTOMOBILE

Event: SALE, REGISTRATION, RENEWAL

Concept: ACCOUNT, COURSE, WORK CENTER

Entity Type / Entity Instance: An **entity type** is a collection of entities that share common properties or characteristics. An **entity instance** is a single occurrence of an entity type.

Strong / Weak Entity Types: A **strong entity type** is one that exists independently of other entity types. E.g. STUDENT, EMPLOYEE, AUTOMOBILE, and COURSE. Instances of a strong entity type always have a unique characteristic (called an *identifier*)—that is, an attribute or a combination of attributes that uniquely distinguish each occurrence of that entity. A **weak entity type** is an entity type whose existence depends on some other entity type. A weak entity type has no business meaning in an E-R diagram without the entity on which it depends. A weak entity type has an attribute that serves as a *partial identifier*. E.g. EMPLOYEE is a strong entity type with identifier Employee ID (we note the identifier attribute by underlining it). DEPENDENT is a weak entity type, as indicated by the double-lined rectangle.



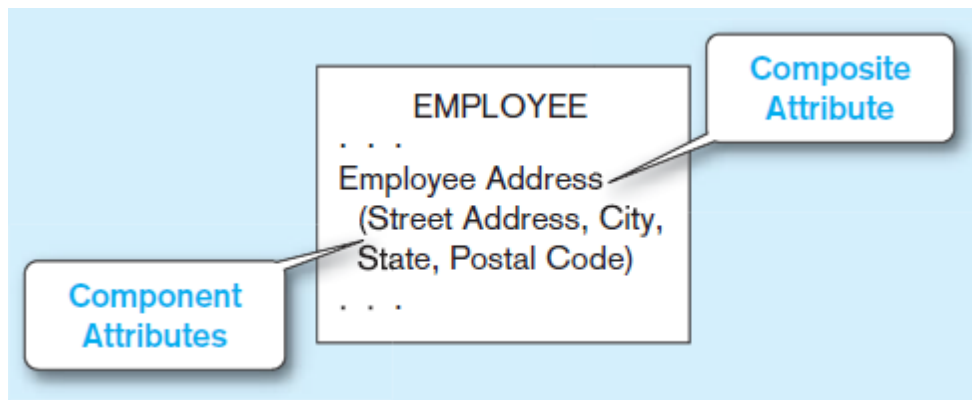
Attributes

An **attribute** is a property or characteristic of an entity type that is of interest to the organization. Thus, an attribute has a noun name. Following are some typical entity types and their associated attributes:

STUDENT	Student ID, Student Name, Home Address, Phone Number, Major
AUTOMOBILE	Vehicle ID, Color, Weight, Horsepower
EMPLOYEE	Employee ID, Employee Name, Payroll Address, Skill

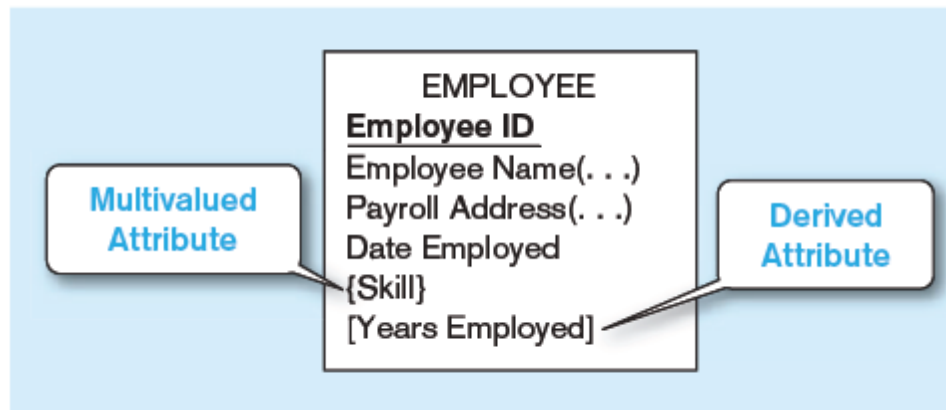
Required / Optional Attributes: An attribute that must be present for each entity instance is called a **required attribute**, whereas an attribute that may not have a value is called an **optional attribute**.

Simple / Composite Attributes: A **composite attribute** is an attribute, such as Address, that has meaningful component parts, which are more detailed attributes. The decision about whether to subdivide an attribute into its component parts depends on whether users will need to refer to those individual components, and hence, they have organizational meaning. A **simple (or atomic) attribute** is an attribute that cannot be broken down into smaller components that are meaningful for the organization.



Single-Valued / Multivalued Attributes: A **multivalued attribute** is an attribute that may take on more than one value for a given entity (or relationship) instance.

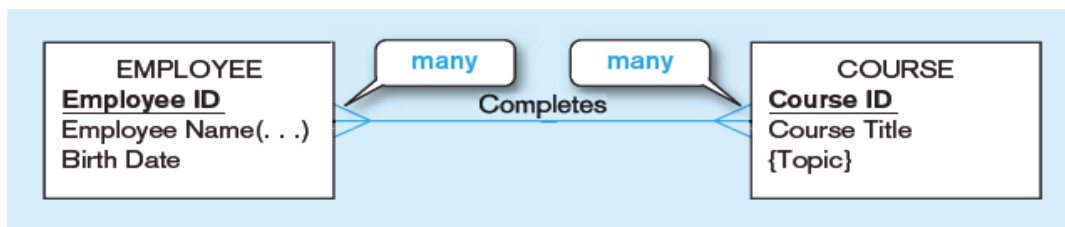
Stored / Derived Attributes: A **derived attribute** is an attribute whose values can be calculated from related attribute values (plus possibly data not in the database, such as today's date, the current time, or a security code provided by a system user). We indicate a derived attribute in an E-R diagram by using square brackets around the attribute name



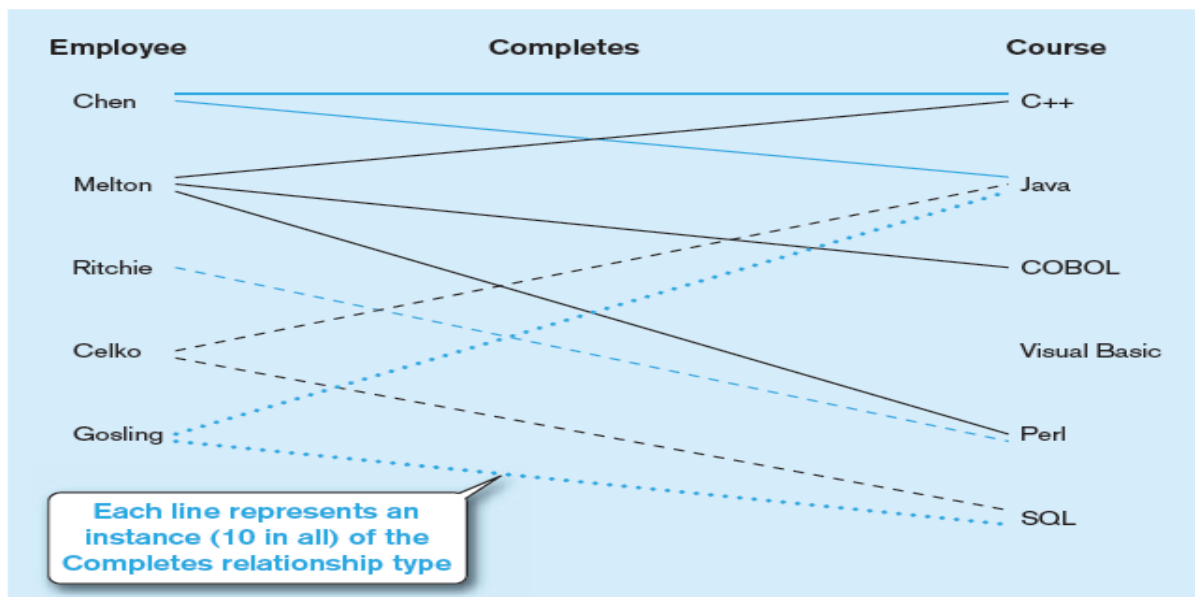
MODELING RELATIONSHIPS

A *relationship* is an association representing an interaction among the instances of one or more entity types that is of interest to the organization.

A **relationship type** is a meaningful association between (or among) entity types. A relationship type is denoted by a line labeled with the name of the relationship.

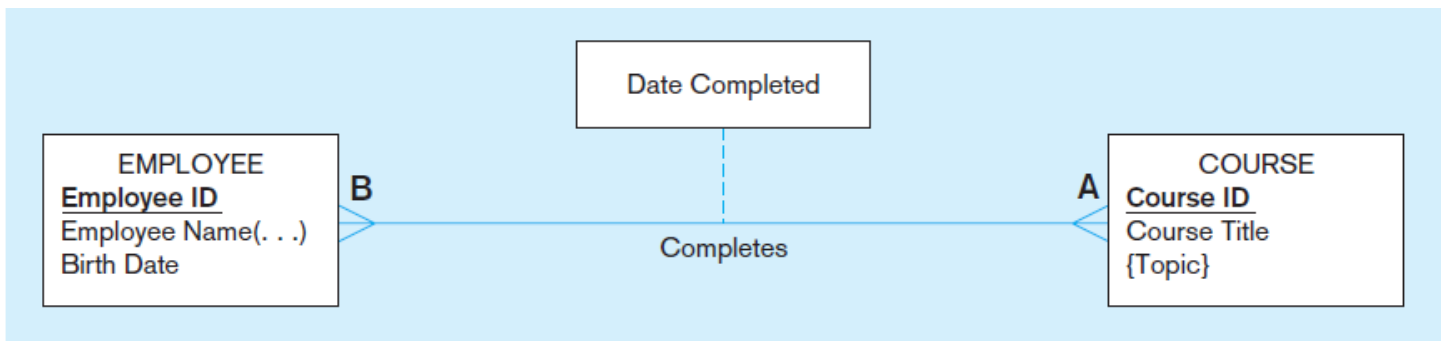


Relationship instance is an association between (or among) entity instances, where each relationship instance associates exactly one entity instance from each participating entity type (Elmasri and Navathe,1994).



ATTRIBUTES ON RELATIONSHIPS

Suppose the organization wishes to record the date (month and year) when an employee completes each course. This attribute is named Date Completed. Date Completed has not been associated with either the EMPLOYEE or COURSE entity. That is because Date Completed is a property of the relationship Completes, rather than a property of either entity. In other words, for each instance of the relationship Completes, there is a value for Date Completed.



ASSOCIATIVE ENTITIES

An **associative entity** is an entity type that associates the instances of one or more entity types and contains attributes that are peculiar to the relationship between those entity instances. The associative entity **CERTIFICATE** is represented with the rectangle with rounded corners.



How do you know whether to convert a relationship to an associative entity type?

Following are four conditions that should exist:

1. All the relationships for the participating entity types are “many” relationships.
2. The resulting associative entity type has independent meaning to end users and, preferably, can be identified with a single-attribute identifier.
3. The associative entity has one or more attributes in addition to the identifier.
4. The associative entity participates in one or more relationships independent of the entities related in the associated relationship.